

★ Libraries

• Numpy

- > It provides efficient storage
- > It also provides better ways of handling data for processing
- > It is fast
- > It is easy to learn
- > Numpy uses relatively less memory to store data.

// Code

Import numpy as np

```
myarr = np.array([[3, 6, 32, 7]], np.int8)
```

```
myarr[0, 1]
```

6 will be output

```
myarr.shape
```

(1, 4) shape

```
myarr.dtype
```

int8 objects

```
myarr[0, 1] = 45
```

at Index 1, value 45 will be

```
myarr
```

Array Creation: Conversion from other Python Structures

```
listarray = np.array([[1, 2, 3], [5, 8, 5], [0, 3, 1]])
```

```
np.array
```

```
listarray.dtype
```

```
listarray.shape
```

```
listarray.size
```

```
np.array([34, 23, 23])
```

Expt. No.:

Initialize numpy array creation objects (e.g., `arange`, `ones`, `zeros`, etc.).

`zeros = np.zeros((2, 5))`

`zeros`

`zeros.shape`

`zeros.dtype`

all arrays are zero

`aranged = np.arange(15)`

`aranged`

`linspace = np.linspace(1, 50, 10)`

`linspace`

diff of two array
index will be equal

`emp = np.empty((4, 6))`

`emp`

form empty array

`ide = np.identity(45)`

`ide`

`ide.shape`

Identity matrix

45 x 45

`arr = np.arange(99)`

`arr`

arrange in sequence
array

`arr = arr.reshape(3, 33)`

`arr`

reshape in (3, 33)

`arr = arr.ravel()`

`arr`

original array return

`arr.shape`

1-D array

Teacher's Signature _____

Numpy Axis

~~arr = np.array(x)~~

x = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

arr = np.array(x)

arr

arr.sum(axis=0)

arr.sum(axis=1)

arr.T

arr.flat

for item in arr.flat:
 print(item)

arr.ndim

arr.size

arr.nbytes

one = np.array([1, 3, 4, 634, 2])

one.argmax()

one.argmin()

one.argmax()

for 2-d Array

arr.argmax(axis=0)

arr.argmin(axis=1)

arr.argmax(axis=1)

arr.argmax(axis=0)

stored all array

([12, 8, 9]) → column

([6, 15, 8]) → row

Transpose of array

If we give all array
in blue form

no of dimension = 2

size of elements

bytes = 36

max element at 3 index

min element at 0 index

sort element with index

max element of axis 0

min element of axis 1

```
arr = np.array([1, 2, 1],
                [4, 0, 6],
                [8, 1, 0])
```

```
arr + arr2
```

```
arr - arr2
```

```
arr * arr2
```

```
np.sqrt(arr)
```

```
np.where(arr > 5)
```

```
np.count_nonzero(arr)
```

square root of array

value which greater than

include 0 in Array

● Pandas / Matplotlib

● Matplotlib

> It is a Python library which is generally used for creating static, animated and interactive plots.

> pyplot is a collection of functions that make matplotlib work as MATLAB.

// Code

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 20, 30, 40, 50]
```

```
plt.plot(x, y)
```

```
plt.show()
```

line chart

display chart in IP

customize chart

```
plt.plot(x, y, color = "green", marker = 'o', linestyle = 'dashed',  
         linewidth = 2, markersize = 12)
```

```
plt.show()
```

```
plt.figure(figsize = (4, 3))
```

figure size

```
plt.plot(x, y, color = "red", marker = 'o', linestyle = 'dashed',  
         linewidth = 2, markersize = 10)
```

```
plt.title("Line chart")
```

```
plt.xlabel("X-axis label")
```

```
plt.ylabel("Y-axis label")
```

```
plt.show()
```

Advanced line chart - Multiple lines & legends

```
x = [1, 2, 3, 4, 5]
```

```
y1 = [20, 40, 60, 80, 100]
```

```
y2 = [10, 30, 50, 70, 90]
```

```

plt.plot(x, y1, label = "Sales 2025") # Plotting y1 data
plt.plot(x, y2, label = "Sales 2026") # Plotting y2 data
plt.title("Sales By Months")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.legend()
plt.show()

```

Bar chart

```

x = ['A', 'B', 'C', 'D', 'E']
y = [10, 20, 30, 40, 50]
plt.figure(figsize=(4, 3))
plt.bar(x, y, color = "pink")
plt.title("Bar chart")
plt.xlabel("X-axis label")
plt.ylabel("Y-axis label")
plt.show()

```

Histogram chart

```

import random
data = [random.randint(1, 50) for _ in range(100)]
plt.hist()
plt.title("Histogram example")
plt.show()

```

Pie chart

```
categories = ['A', 'B', 'C', 'D', 'E']  
sales = [10, 20, 55, 35, 45]  
plt.pie(sales, labels=categories, autopct = '%.1f%%')  
plt.title("Pie chart")  
plt.show()
```

Scatter plot

> generally used for finding relationship b/w two variables

data

```
y1 = [10, 20, 30, 40, 50]
```

```
y2 = [20, 40, 60, 80, 100]
```

```
plt.scatter(y1, y2)
```

```
plt.xlabel("y1 - axis label")
```

```
plt.ylabel("y2 - axis label")
```

```
plt.title("Scatter plot")
```

```
plt.show()
```

Subplots

> It generally used to show multiple charts in one figure

data 1

```
categories = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']
```

```
Sales = [10, 20, 30, 40, 50]
```

data 2

```
y1 = [10, 20, 30, 40, 50]
```

```
y2 = [30, 40, 80, 90, 100]
```

```
plt.figure(figsize=(10,6))
```

first Plot - bar chart

plt.subplot (1, 2, 1)

row, column, position

plt.bar (categories, sales)

plt.title ('Weekly sales')

plt.xlabel ('Week days')

plt.ylabel ('Sales')

second plot - Scatter Plot

plt.subplot (1, 2, 2)

plt.scatter (y1, y2)

plt.title ("Daily sales")

plt.xlabel ("User 1")

plt.ylabel ("User 2")

plt.show ()

Machine Learning Tutorials

Phase 1:

★ Data cleaning & Feature Engineering

● Data cleaning

↳ Data cleaning is the process of detecting incomplete, inconsistent, inaccurate or irrelevant parts of the data and then replacing, modifying or deleting the dirty data.

● Steps of Data cleaning

↳ Import datasets

↳ merge datasets into one datasets

↳ identifying error

↳ standardize process

↳ Scrub for duplicates

↳ validate accuracy

● Methods of Handling missing values

↳ What is missing value/data?

In the variable or any observation, values are not stored called as missing values/data.

Age	Sex	Male	Female	Age range
23	Male	1	0	21-30
12	Female	0	1	-
34	Male	1	0	31-40
26	-	0	1	21-30
-	Male	1	0	11-20
38	Male	1	0	31-40

- a) Methods to Handling Missing Values
- b) Ignore missing values row/ Delete row
- c) Fill missing value manually
- d) Global constant

- a) Measure of Central tendency (Mean, median & mode)
- b) Measure of Central tendency for each class
- c) Most probable value (such as ML Algorithm)

// Code

```
#import libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# load datasets
```

```
df = pd.read_csv(r"C:\User Singh\Downloads\house-prices-advanced-regression-techniques\train.csv")
```

```
df.shape
```

```
df.head(10)
```

```
df.tail(10)
```

```
df.info()
```

```
pd.set_option('display.max_columns', None)
```

```
pd.set_option('display.max_rows', None)
```

```
df.head(10)
```

```
df.tail(10)
```

```
df.info()
```

```
df.isnull().sum()
```

```
plt.figure(figsize=(25,25))
```

```
sns.heatmap(df.isnull())
```

```
null-var = df.isnull().sum() / df.shape[0] * 100
```

```
null-var
```

```
drop_columns = null-var[null-var > 15].keys()
```

```
drop_columns
```

```
df2-drop_columns = df.drop(columns=drop_columns)
```

```
df2-drop_columns
```

```
sns.heatmap(df2-drop_columns.isnull())
```

```
df3-drop_rows = df2-drop_columns.dropna()
```

```
df3-drop_rows.shape
```

```
df3-drop_rows sns.heatmap(df3-drop_rows.isnull())
```

```
df3-drop_rows.isnull().sum()
```

```
df3-drop_rows.select_dtypes(include=['int64', 'float64']).columns
```

```
sns.distplot(df['OverallQual'])
```

```
sns.distplot(df3-drop_rows['OverallQual'])
```

```
num-var = ["all integer and float value"]
```

```
plt.figure(figsize=(25,25))
```

```
for i, var in enumerate(num-var):
```

```
    plt.subplot(9,4,i+1)
```

```
    sns.distplot(df[var], bins=20)
```

```
    sns.distplot(df3-drop_rows[var], bins=20)
```

★ Missing value of Central Tendency (Mean, Median, Mode) (mean)

```
missing-num-var = [var for var in df3-num.columns
                    if df3-num[var].isnull().sum() > 0]
```

```
missing-num-var
```

```
plt.figure(figsize=(10,10))
```

```
sns.set()
```

```
for i, var in enumerate(missing-num-var):
```

```
    plt.subplot(2,2,i+1)
```

```
    sns.distplot(df3-num[var], bins=20, kde_kws =
                  {'linewidth': 5, 'color': '#DC143C'})
```

```
df3-num
```

```
df4-num-mean = df3-num.fillna(df3-num.mean())
```

```
df4-num-mean.isnull().sum()
```

```
plt.figure(figsize=(10,10))
```

```
sns.set()
```

```
for i, var in enumerate(missing-num-var):
```

```
    plt.subplot(2,2,i+1)
```

```
    sns.distplot(df3-num[var], bins=20, kde_kws =
                  {'linewidth': 8, 'color': 'red',
                   label = "Original"})
```

```
    sns.distplot(df4-num-mean[var], bins=20, kde_kws =
                  {'linewidth': 5, 'color': 'green',
                   label = "Mean"})
```

```
plt.legend()
```

```
plt.legend()
```

2. Pandas Libraries

★ Code

```
[1]: import numpy as np  
import pandas as pd
```

```
[2]: dict1 = {  
    "name": ['Mahip', 'Mohit', 'Ayush', 'Aditya', 'Chinmay'],  
    "marks": [92, 85, 76, 65, 55],  
    "College": ['smvit', 'ait', 'nmit', 'bmsit', 'rvce'],  
    "State": ['bihar', 'bihar', 'Odisha', 'Kerala', 'Karnataka']  
}
```

```
[3]: df = pd.DataFrame(dict1)
```

```
[4]: df
```

Teacher's Signature _____

```

[5]: df.to_csv('college_results.csv') # It create one file (csv)
[6]: df.to_csv('college-index-false.csv', index=False) # Index delete
[7]: df.head(2) # Starting 2 data from head
[8]: df.tail(2) # Starting 2 data from tail
[9]: df.describe() # Statistics calculation
[10]: Train=pd.read_csv('Train-Tracking.csv')
[11]: Train
[12]: Train['Arrival City'][0] = 'Bihar'
[13]: Train
[14]: Train.index=['1st', '2nd', '3rd', '4th', '5th', '6th', '7th']
[15]: Train

```

★ What is Pandas?

- > Pandas is an open source data analysis library written in python.
- > It leverages the power and speed of numpy to make data analysis and preprocessing easy for data scientists.
- > It provides rich and highly robust data operation.

★ Pandas Data Structure

- > Pandas has two types of data structures:
 - a) Series: It's one dimensional array with indices, It stores a single column or row of data in a Dataframe.
 - b) Dataframe: It's a tabular spreadsheet like structure representing rows each of contains one or more columns.

- > A one-dimensional Array (labeled) capable of holding any type of data - Series.
- > A two-dimensional Array (labeled) structures with columns of potentially different types of data - DataFrame.

● Code

```
[1]: Ser = pd.Series(np.random.rand)
[2]: Ser
[3]: Ser = pd.Series(np.random.rand(40))
[4]: Ser
[5]: type(Ser)
[6]: newdf = pd.DataFrame(np.random.rand(100, 5), index =
                        np.arange(100))
[7]: newdf
[8]: newdf.head()
[9]: newdf.tail()
[10]: type(newdf)
[11]: newdf.describe()
[12]: newdf.dtypes
[13]: newdf[0][0] = "Malip"
[14]: newdf.head()
[15]: newdf.dtypes
[16]: newdf.T
[17]: newdf.sort_index(axis=0, ascending=False)
[18]: newdf.sort_index(axis=1, ascending=False)
[19]: newdf1 = newdf
```

[20]: newdf[0][0] = "Mahit"

[21]: newdf

[22]: newdf = newdf.copy()

[23]: newdf[0][0] = "Mahip"

[24]: newdf

[25]: newdf.loc[0,0] = 0.12345

[26]: newdf.head()

[27]: newdf.columns = list("ABCDE")

[28]: newdf.head()

[29]: newdf.loc[0,0] = 0.123

[30]: newdf

[31]: newdf.drop(0, axis=1)

[32]: newdf.loc[[1,2], ['D', 'E']]